

DumpUtility

Complete Codebase Explanation

For .NET / C# Beginners -- Syntax, Logic & Data Flow

What this document covers:

- * Every file and every line -- syntax, logic and purpose explained
- * How data flows from JSON through processing to output folders
- * How test cases work and what each test verifies
- * How to swap dummy data for a real SQL Server View (future integration)
- * Architecture, naming rules, logging and merge behaviour

Table of Contents

1. Project Overview & Purpose
2. Folder & File Structure
3. End-to-End Architecture Diagram
4. Data Flow Walkthrough (with example data)
5. Models -- the data shapes
 - 5.1 DocumentRecord.cs
 - 5.2 AppConfig.cs
 - 5.3 DatabaseConfig.cs
6. Helpers -- utility tools
 - 6.1 LogHelper.cs
 - 6.2 FileNamingHelper.cs
7. Services -- the business logic
 - 7.1 IDataService.cs (interface)
 - 7.2 ConfigService.cs
 - 7.3 DummyDataService.cs
 - 7.4 FolderService.cs (core engine)
 - 7.5 PdfMergeService.cs
 - 7.6 ReportService.cs
8. Program.cs -- the entry point
9. Test Cases -- PdfMergeServiceTests.cs
10. Database View Integration (Future)
11. Glossary of C# / .NET terms used

1. Project Overview & Purpose

DumpUtility is a .NET 8 console application written in C#. Its job is to take a list of employee HR documents and automatically organise them into a structured folder hierarchy on disk -- one folder per employee, with sub-folders that match the document category (e.g. '1.1 CV Signed', '1.4 Photograph').

Every time the program runs it creates a fresh timestamped output folder called 'Dump_Utility_<DDMMYYYY_HHmm>' so previous runs are never overwritten. It also produces a plain-text log file and a summary file.

PDF and image files in the same folder-category are automatically MERGED into a single PDF. Non-image/PDF files (like .docx) are copied individually with sequential names.

Why .NET / C#?

.NET is Microsoft's free, cross-platform runtime. C# is its primary language. It compiles to native machine code, runs fast, and has a huge standard library. The project targets .NET 8 (the current Long-Term Support release as of 2024).

The app uses the PdfSharp library (open source) for PDF merging and image embedding. Everything else is pure .NET standard library.

Key concepts a beginner needs to know

Namespace -- A named container for classes -- like a folder for code. Prevents name collisions across big projects.

Class -- A blueprint for objects. Contains fields (data) and methods (actions).

Interface -- A contract: 'any class implementing this must have these methods.' Enables swapping implementations.

Method -- A named block of code that does one thing. Called by name. Like a function in Python/JS.

Property -- A class member that looks like a variable but can have get/set logic behind it.

using -- Imports a namespace so you can use its classes without typing the full path.

var -- Tells the compiler to infer the type automatically. Like 'let' in JavaScript.

string -- Text. Immutable sequence of characters. C# strings use double quotes.

List<T> -- A dynamic array. T is the element type, e.g. List<string> is a list of strings.

Dictionary<K,V> -- A key->value map. Like Python's dict or JavaScript's object.

IEnumerable<T> -- Anything you can iterate over with foreach. Lazily evaluated.

async/await -- Marks code that runs without blocking the thread -- not used in this project yet but common in .NET.

2. Folder & File Structure

Below is the complete layout of the repository. Understanding where each file lives helps you know where to look when a feature needs changing.

```
AutoFileSys/          <- git repository root
  DumpUtility/        <- the main C# project
    Program.cs        <- entry point (top-level statements)
    DumpUtility.csproj <- project file (NuGet packages, target framework)
    config.json        <- runtime configuration (BaseFolder path)
    Models/
      AppConfig.cs     <- shape of config.json
      DatabaseConfig.cs <- shape of the DB section in config.json
      DocumentRecord.cs <- one HR document record (4 fields)
    Services/
      IDataService.cs  <- interface: contract for data providers
      ConfigService.cs  <- reads and validates config.json
      DummyDataService.cs <- reads records from records.json file
      FolderService.cs  <- CORE: creates folders, copies/merges files
      PdfMergeService.cs <- merges PDFs and images into one PDF
      ReportService.cs  <- writes the Summary_*.txt file
    Helpers/
      LogHelper.cs      <- thread-safe append-only logger
      FileNamingHelper.cs <- pure naming/sanitisation rules
    dummy_data/
      records.json      <- sample data (original, 1 employee)
      records2.json     <- sample data (2 employees, richer)
      pdf/              <- sample PDF files
      doc/              <- sample DOCX files
      img/              <- sample image files (.jif, .webp, .jpg)
    DumpUtility.Tests/  <- xUnit test project
    Services/
      PdfMergeServiceTests.cs <- tests for PdfMergeService
```

What gets produced at runtime

After 'dotnet run', you will find the following in the BaseFolder (e.g. C:\DumpOutput):

```
C:\DumpOutput\
  Log_26052026_1430.txt      <- timestamped log of every action
  Summary_26052026_1430.txt  <- human-readable summary
  Dump_Utility_26052026_1430\ <- the main output run folder
    0123-Rishikesh\
      1. Documents for Onboarding\
        1.1 CV Signed\
          CV Signed_0123-Rishikesh.pdf <- merged PDF (pdf+img)
          CV Signed_0123-Rishikesh_1.docx <- copied non-eligible file
        1.3 Offer letter - acknowledged\
          ...
    0456-Aamir\
      ...
```

3. End-to-End Architecture Diagram

The diagram below shows how the classes connect and what each layer is responsible for. Read it top-to-bottom.



Dependency Injection (manual)

FolderService needs a LogHelper and a PdfMergeService. These are created in Program.cs and passed in through the constructor. This is called manual dependency injection -- instead of the class creating its own dependencies, they are 'injected' from outside. This makes testing easy: in tests you can pass a test logger or mock service.

4. Data Flow Walkthrough (with Example Data)

Let's trace a single real example through the whole system using the first two records from records2.json for employee Rishikesh (EmployeeId = 123):

```
{ "EmployeeId": 123, "EmployeeName": "Rishikesh",  
  "FolderPath": "1. Documents for Onboarding/1.1 CV Signed",  
  "FilePath": "dummy_data/pdf/rishikesh.pdf" }  
  
{ "EmployeeId": 123, "EmployeeName": "Rishikesh",  
  "FolderPath": "1. Documents for Onboarding/1.1 CV Signed",  
  "FilePath": "dummy_data/doc/rishikesh.docx" }
```

Step 1 -- Config loaded

ConfigService reads config.json. It finds BaseFolder = 'C:\DumpOutput'. The app creates that directory if it doesn't exist.

Step 2 -- Records loaded

DummyDataService reads records2.json and deserialises it into a List<DocumentRecord>. The two records above become two C# objects in memory with four properties each.

Step 3 -- Run folder created

Program.cs builds the name 'Dump_Utility_26052026_1430' and asks FolderService to use that as the root output folder.

Step 4 -- Group by EmployeeId

FolderService groups all records by EmployeeId. For EmployeeId=123 it gets all 12 records for Rishikesh. It then calls GetEmployeeFolder(123, 'Rishikesh') which returns '0123-Rishikesh'.

Step 5 -- Employee folder created

Directory 'C:\DumpOutput\Dump_Utility_26052026_1430\0123-Rishikesh\' is created.

Step 6 -- Group by FolderPath

Within Rishikesh's records, they are grouped by FolderPath. The two records above share FolderPath = '1. Documents for Onboarding/1.1 CV Signed'.

Step 7 -- Sub-folder tree created

BuildFullPath splits '1. Documents for Onboarding/1.1 CV Signed' on '/' to get two segments: ['1. Documents for Onboarding', '1.1 CV Signed']. Each becomes a sub-directory. SanitizeSegment replaces any invalid chars (none here). The full destination folder is: '...\0123-Rishikesh\1. Documents for Onboarding\1.1 CV Signed\'.

Step 8 -- Eligibility split

The two records are split: rishikesh.pdf has extension .pdf -> ELIGIBLE for merge. rishikesh.docx has extension .docx -> NOT eligible, copy individually.

Step 9 -- Non-eligible file copied (Pass 1)

rishikesh.docx is copied to destination with name built by BuildFileName: 'CV Signed_0123-Rishikesh_1.docx'. The sequence number (1) increments for each additional non-eligible file in the same folder.

Step 10 -- Eligible files merged (Pass 2)

rishikesh.pdf is the only eligible file for this folder. PdfMergeService.Merge(['...\rishikesh.pdf'], '...\CV Signed_0123-Rishikesh.pdf') is called. The result is a merged PDF named by BuildMergedFileName.

Step 11 -- FolderResult updated

FolderResult.FilesWritten += 2, MergedGroups += 1, FilesProcessedInMerge += 1. The same process repeats for every other folder group and employee.

Step 12 -- Summary written

ReportService.WriteSummary writes Summary_26052026_1430.txt with counts and the list of employee paths.

5. Models -- The Data Shapes

Models are simple data containers -- they hold data but don't perform actions. In C# these are called POCOs (Plain Old CLR Objects). Think of them like TypeScript interfaces or Python dataclasses.

5.1 DocumentRecord.cs

```
namespace DumpUtility.Models;

public class DocumentRecord
{
    public int EmployeeId { get; set; } = 0;
    public string EmployeeName { get; set; } = string.Empty;
    public string FolderPath { get; set; } = string.Empty;
    public string FilePath { get; set; } = string.Empty;
}
```

Line: namespace DumpUtility.Models;

Declares that this class lives inside the 'DumpUtility.Models' namespace. Any other file can use it by writing 'using DumpUtility.Models;' at the top.

Line: public class DocumentRecord

'public' means other files can see this class. 'class' declares a new class (a blueprint for objects). 'DocumentRecord' is the name.

Line: public int EmployeeId { get; set; } = 0;

'int' is a 32-bit integer. '{ get; set; }' is an auto-property -- the compiler creates a hidden backing field automatically. '= 0' is the default value if none is provided. This corresponds to the 'EmployeeId' key in the JSON.

Line: public string EmployeeName { get; set; } = string.Empty;

'string' is text. 'string.Empty' is an empty string ("") -- a safe default so the property is never null.

Line: public string FolderPath { get; set; } = string.Empty;

Stores a path like '1. Documents for Onboarding/1.1 CV Signed'. The slash separator is parsed later in FolderService.

Line: public string FilePath { get; set; } = string.Empty;

Relative or absolute path to the source file on disk, e.g. 'dummy_data/pdf/rishikesh.pdf'.

5.2 AppConfig.cs

```
namespace DumpUtility.Models;

public class AppConfig
{
    public string BaseFolder { get; set; } = string.Empty;
    public DatabaseConfig Database { get; set; } = new();
}
```


'BaseFolder' maps to the JSON key 'BaseFolder' -- the root output directory. 'Database' is a nested object of type DatabaseConfig, also from the JSON. '= new()' is shorthand for '= new DatabaseConfig()' -- C# 9+ target-typed new expression. This ensures Database is never null.

5.3 DatabaseConfig.cs

```
public class DatabaseConfig
{
    public string Server      { get; set; } = string.Empty;
    public string DatabaseName { get; set; } = string.Empty;
    public string Username    { get; set; } = string.Empty;
    public string Password    { get; set; } = string.Empty;
    public int    Port        { get; set; } = 1433;
}
```

This holds future SQL Server connection settings. '1433' is SQL Server's default TCP port. Currently these fields are loaded from config.json but not used -- the database integration is planned for a later phase. Program.cs checks config.Database.Server only to log whether a DB config is present.

6. Helpers -- Utility Tools

6.1 LogHelper.cs

LogHelper provides a thread-safe, append-only file logger with three severity levels: INFO, WARN, ERROR. 'Thread-safe' means it can be called from multiple threads simultaneously without corrupting the log file.

```
namespace DumpUtility.Helpers;

public class LogHelper
{
    private readonly string _logFilePath;
    private readonly object _lock = new();

    public LogHelper(string logFilePath)
    {
        _logFilePath = logFilePath;
        Directory.CreateDirectory(Path.GetDirectoryName(logFilePath)!);
    }

    public void Info(string message) => Write("INFO ", message);
    public void Warn(string message) => Write("WARN ", message);
    public void Error(string message) => Write("ERROR", message);

    private void Write(string level, string message)
    {
        string entry = $"[{DateTime.Now:yyyy-MM-dd HH:mm:ss}] [{level}] {message}";
        lock (_lock)
        {
            File.AppendAllText(_logFilePath, entry + Environment.NewLine);
        }
    }
}
```

private readonly string _logFilePath;

'private' -- only visible inside this class. 'readonly' -- can only be set in the constructor, never changed after. Think of it as a final field in Java or a frozen variable.

private readonly object _lock = new();

An opaque 'lock token' object used to synchronise threads. When one thread holds the lock, other threads wait. 'new()' is the same as 'new object()'.

Directory.CreateDirectory(Path.GetDirectoryName(logFilePath)!);

'Path.GetDirectoryName' extracts the folder part of a path. 'Directory.CreateDirectory' creates it (and all parent folders) if it does not exist. The '!' at the end is the null-forgiving operator -- it tells the compiler 'I know this is not null, trust me'. It silences a nullable warning.

public void Info(...) => Write("INFO ", message);

Expression-bodied method -- a shorthand for a one-liner method body. The '=>' means 'this method returns / does the following'. These three lines are convenience wrappers that call the private Write method.

string entry = \$"[{DateTime.Now:...}] [{level}] {message}";

A C# interpolated string (prefix '\$'). Curly braces embed expressions. 'DateTime.Now:yyyy-MM-dd HH:mm:ss' formats the current date/time. The result looks like: '[2026-05-26 14:30:00] [INFO] App started'.

lock (_lock) { File.AppendAllText(...); }

'lock' acquires the monitor on _lock. Only one thread can be inside the lock block at a time. 'File.AppendAllText' opens the file, appends the text, and closes it -- all in one call. 'Environment.NewLine' is '\r\n' on Windows, '\n' on Linux.

6.2 FileNamingHelper.cs

FileNamingHelper is a static utility class -- it has no instance state, only pure functions. It handles all the rules for constructing file and folder names so the logic is in one place.

```
public static class FileNamingHelper
{
    private static readonly char[] InvalidChars =
        ['/', ':', '*', '?', '"', '<', '>', '|'];
    private static readonly Regex NumericPrefix =
        new(@"^\d+(\.\d+)*\s+", RegexOptions.Compiled);

    public static string GetEmployeeFolder(int employeeId, string employeeName)
    {
        string paddedId = employeeId < 10000
            ? employeeId.ToString().PadLeft(4, '0')
            : employeeId.ToString();
        return $"{paddedId}-{SanitizeSegment(employeeName)}";
    }

    public static string GetSubcategoryName(string folderPath)
    {
        string lastSegment = folderPath
            .Replace('/', Path.DirectorySeparatorChar)
            .Split(Path.DirectorySeparatorChar, StringSplitOptions.RemoveEmptyEntries)
            .Last();
        return NumericPrefix.Replace(lastSegment, string.Empty);
    }

    public static string SanitizeSegment(string segment) =>
        string.Concat(segment.Select(c => InvalidChars.Contains(c) ? '_' : c));

    public static string BuildFileName(string subcategoryName,
        string employeeFolder, int sequence, string extension) =>
        $"{SanitizeSegment(subcategoryName)}_{SanitizeSegment(employeeFolder)}_"
        + $"{sequence}{extension}";

    public static string BuildMergedFileName(string subcategoryName,
        string employeeFolder) =>
        $"{SanitizeSegment(subcategoryName)}_{SanitizeSegment(employeeFolder)}.pdf";
}
```

private static readonly char[] InvalidChars = [...]

A compile-time constant array of characters that are illegal in Windows file names. 'static' means it belongs to the class, not to any instance. 'readonly' means the reference cannot be reassigned. The literal syntax '['...'']' is a C# 12 collection expression.

private static readonly Regex NumericPrefix = new(@"^\d+(\.\d+)*\s+", ...)

A compiled regular expression. '\d+' matches one or more digits. '(\.\d+)*' matches optional '.1', '.2' etc. '\s+' matches one or more spaces. The whole pattern matches prefixes like '1. ', '1.1 ', '9.2.1 ' at the START of a string (the '^' anchor). RegexOptions.Compiled compiles it to machine code for speed.

employeeId < 10000 ? ... PadLeft(4, '0') : ...

Ternary operator: if condition ? value_if_true : value_if_false. For id=123: '123'.PadLeft(4,'0') = '0123'. For id=10001: '10001' (no padding needed).

return \$"{paddedId}-{SanitizeSegment(employeeName)}";

Produces e.g. '0123-Rishikesh'. The colon and slash in the name would be sanitised if present.

folderPath.Replace('/', ...).Split(...).Last()

Normalise separators, split into segments, take the last one. For '1. Documents for Onboarding/1.1 CV Signed' the last segment is '1.1 CV Signed'.

NumericPrefix.Replace(lastSegment, string.Empty)

Strips the leading '1.1 ' part, leaving 'CV Signed'. This becomes the subcategory name used in output file names.

string.Concat(segment.Select(c => InvalidChars.Contains(c) ? '_' : c))

'Select' is LINQ -- it maps each character to either '_' (if invalid) or itself. 'string.Concat' joins the characters back into a string. Pure functional style.

BuildFileName(subcategoryName, empFolder, seq, ext)

Produces names like 'CV Signed_0123-Rishikesh_1.docx'. The sequence number distinguishes multiple non-eligible files in the same folder category.

BuildMergedFileName(subcategoryName, empFolder)

Produces 'CV Signed_0123-Rishikesh.pdf' -- no sequence, no extension param, always .pdf because it is the merged output.

7. Services -- The Business Logic

7.1 IDataService.cs (Interface)

```
using DumpUtility.Models;

namespace DumpUtility.Services;

public interface IDataService
{
    IEnumerable<DocumentRecord> GetRecords();
}
```

An interface in C# is a pure contract -- it only says WHAT a class must do, never HOW. Any class that writes 'class Foo : IDataService' must implement 'GetRecords()' or it won't compile.

'IEnumerable<DocumentRecord>' is a lazy sequence. The method can return a List, an array, a database cursor -- anything iterable. Program.cs calls '.ToList()' to materialise it into memory.

Why use an interface here?

Today the data comes from a JSON file (DummyDataService). Soon it will come from a SQL Server View (SqlDataService). Because Program.cs uses IDataService instead of the concrete class, you only need to write a new SqlDataService that implements IDataService -- the rest of the code never changes.

7.2 ConfigService.cs

```
using System.Text.Json;
using DumpUtility.Models;

namespace DumpUtility.Services;

public class ConfigService
{
    public AppConfig Load(string configPath)
    {
        if (!File.Exists(configPath))
            throw new FileNotFoundException($"config.json not found at: {configPath}");

        string json = File.ReadAllText(configPath);
        var config = JsonSerializer.Deserialize<AppConfig>(json,
            new JsonSerializerOptions { PropertyNameCaseInsensitive = true });

        if (config is null)
            throw new InvalidOperationException("Failed to deserialize config.json.");

        if (string.IsNullOrEmpty(config.BaseFolder))
            throw new InvalidOperationException("BaseFolder is required.");

        return config;
    }
}
```

using System.Text.Json;

Imports the .NET built-in JSON library. No NuGet package needed -- it ships with the runtime since .NET 5.

if (!File.Exists(configPath)) throw ...

'File.Exists' returns true/false. 'throw' stops execution and propagates the exception up the call stack. 'FileNotFoundException' is a standard .NET exception type.

string json = File.ReadAllText(configPath);

Reads the entire file into a string. Blocks until complete. Fine for config files which are always small.

JsonSerializer.Deserialize<AppConfig>(json, new JsonSerializerOptions { ... })

'Deserialize<AppConfig>' means: parse the JSON and map keys to properties of AppConfig. 'PropertyNameCaseInsensitive = true' means 'basefolder', 'BaseFolder' and 'BASEFOLDER' all map to the same property.

if (config is null)

C# pattern matching. 'is null' is safer than '== null' for reference types. If the JSON was empty or malformed, Deserialize returns null.

if (string.IsNullOrEmpty(config.BaseFolder))

Validates the loaded config. IsNullOrEmpty returns true for null, empty string, or strings containing only spaces/tabs. This ensures the app won't silently write files to a wrong location.

7.3 DummyDataService.cs

```
public class DummyDataService : IDataService
{
    private readonly string _recordsPath;

    public DummyDataService(string recordsPath) => _recordsPath = recordsPath;

    public IEnumerable<DocumentRecord> GetRecords()
    {
        if (!File.Exists(_recordsPath))
            throw new FileNotFoundException(...);

        string json = File.ReadAllText(_recordsPath);
        return JsonSerializer.Deserialize<List<DocumentRecord>>(json,
            new JsonSerializerOptions
            {
                PropertyNameCaseInsensitive = true,
                ReadCommentHandling = JsonCommentHandling.Skip,
                AllowTrailingCommas = true
            }) ?? [];
    }
}
```

public class DummyDataService : IDataService

The colon ':' means 'implements'. This class promises to provide all methods declared in IDataService. The compiler enforces this.

public DummyDataService(string recordsPath) => _recordsPath = recordsPath;

Constructor -- runs when 'new DummyDataService(path)' is called. The '=>' expression body assigns the parameter to the field. This is a common one-liner constructor pattern in modern C#.

ReadCommentHandling = JsonCommentHandling.Skip

Tells the JSON parser to ignore '/' and '/* */' style comments. Standard JSON does not allow comments but this option adds tolerance.

AllowTrailingCommas = true

JSON strictly forbids a comma after the last element. This option relaxes that rule so developers can add records without worrying about removing the last comma.

?? []

The null-coalescing operator. If Deserialize returns null (impossible here but the compiler doesn't know that), use '[]' -- an empty list. C# 12 collection expression syntax.

7.4 FolderService.cs -- The Core Engine

FolderService is the most important class in the project. It takes the list of records and produces the entire folder structure. Let's walk through every section.

7.4.1 FolderResult -- the return type

```
public class FolderResult
{
    public int FoldersCreated      { get; set; }
    public int FilesWritten        { get; set; }
    public int MergedGroups        { get; set; }
    public int FilesProcessedInMerge { get; set; }
    public List<(string FilePath, string Error)> SkippedFiles { get; } = [];
    public Dictionary<string, string> EmployeePaths { get; } = [];
}
```

'List<(string FilePath, string Error)>' is a list of C# VALUE TUPLES. Each element is a pair of two strings. The names FilePath and Error are just labels for readability -- they don't add overhead. This avoids creating a dedicated class just to hold two strings.

'Dictionary<string, string>' maps employee folder name (key) to its full path (value). e.g. '0123-Rishikesh' -> 'C:\DumpOutput\Dump_Utility_...\0123-Rishikesh'.

Notice SkippedFiles and EmployeePaths use '{ get; }' -- a get-only property. The collection itself can be modified (you can Add items) but the property reference cannot be replaced with a different list. This is a common pattern for collection properties.

7.4.2 The Execute method -- line by line

```
public FolderResult Execute(string runFolderPath,
    IEnumerable<DocumentRecord> records, string appBaseDir)
{
    var result = new FolderResult();
    EnsureFolder(runFolderPath, result);

    var byEmployee = records
        .GroupBy(r => r.EmployeeId)
        .OrderBy(g => g.Key);

    foreach (var employeeGroup in byEmployee)
    {
        var first = employeeGroup.First();
        string empFolder = FileNamingHelper.GetEmployeeFolder(
            first.EmployeeId, first.EmployeeName);
        string empPath = Path.Combine(runFolderPath, empFolder);
        EnsureFolder(empPath, result);
        result.EmployeePaths[empFolder] = empPath;

        int filesBeforeEmployee = result.FilesWritten;

        foreach (var folderGroup in employeeGroup.GroupBy(r => r.FolderPath))
        {
            ... (see next section)
        }
        Console.WriteLine($"  Processed: {empFolder} ...");
    }
}
```

```

    }
    return result;
}

```

var result = new FolderResult();

Create the result object that will accumulate counts. It starts with zeroes and empty collections.

EnsureFolder(runFolderPath, result)

Create the run folder (e.g. 'Dump_Utility_26052026_1430') if it doesn't exist, incrementing result.FoldersCreated.

records.GroupBy(r => r.EmployeeId)

LINQ GroupBy. Takes a flat list and groups elements with the same EmployeeId. 'r' is the loop variable (a DocumentRecord). The result is a sequence of groups, each group containing all records for one employee.

.OrderBy(g => g.Key)

Sorts groups by their key (EmployeeId) ascending. Ensures output folders are always created in ID order, making output predictable.

var first = employeeGroup.First();

Gets the first record in the group to extract the employee name. All records in a group have the same EmployeeId so any one of them works for the name.

int filesBeforeEmployee = result.FilesWritten;

Snapshot the current file count so we can compute how many files were written for THIS employee specifically (used in the Console.WriteLine below).

int filesForEmployee = result.FilesWritten - filesBeforeEmployee;

The difference gives the files produced for this employee. Simple subtraction on the two snapshots.

7.4.3 Inside the FolderPath loop

```

string destFolder = BuildFullPath(empPath, folderGroup.Key, result);
string subCategory = FileNamingHelper.GetSubcategoryName(folderGroup.Key);

var nonEligible = folderGroup.Where(r => !PdfMergeService.IsEligible(r.FilePath)).ToList();
var eligible    = folderGroup.Where(r => PdfMergeService.IsEligible(r.FilePath)).ToList();

// Pass 1: copy non-eligible files with sequence names
int seq = 0;
foreach (var record in nonEligible)
{
    seq++;
    string ext          = Path.GetExtension(record.FilePath);
    string destFileName = FileNamingHelper.BuildFileName(subCategory, empFolder, seq, ext);
    string destPath     = Path.Combine(destFolder, destFileName);
    string sourcePath   = ResolveSourcePath(record.FilePath, appBaseDir);
    File.Copy(sourcePath, destPath, overwrite: true);
    result.FilesWritten++;
}

// Pass 2: merge eligible files into one PDF

```

```

if (eligible.Count == 0) continue;
var resolvedPaths = new List<string>();
foreach (var record in eligible)
{
    string sourcePath = ResolveSourcePath(record.FilePath, appBaseDir);
    if (!File.Exists(sourcePath)) { /* log and skip */ }
    else resolvedPaths.Add(sourcePath);
}
string mergedFileName = FileNamingHelper.BuildMergedFileName(subCategory, empFolder);
string mergedPath      = Path.Combine(destFolder, mergedFileName);
_mergeService.Merge(resolvedPaths, mergedPath);
result.FilesWritten++;
result.MergedGroups++;

```

BuildFullPath(empPath, folderGroup.Key, result)

Creates the nested sub-directory tree for this category. folderGroup.Key is the FolderPath string like '1. Documents for Onboarding/1.1 CV Signed'.

folderGroup.Where(r => !PdfMergeService.IsEligible(r.FilePath)).ToList()

LINQ Where filters records to those whose file is NOT eligible for merging (i.e. not PDF/image). '.ToList()' materialises the lazy query into a concrete List so Count and iteration work efficiently.

overwrite: true

Named argument. If the destination file already exists, replace it. Without this, File.Copy throws if the destination exists. Named arguments improve readability -- you can tell what 'true' means without looking at the signature.

result.FilesWritten++

Post-increment. 'x++' reads x, returns x, then adds 1. '++x' adds 1 first. Here it doesn't matter which form since we're not using the return value.

if (eligible.Count == 0) continue;

'continue' jumps to the next iteration of the enclosing foreach. This skips the merge entirely if there are no eligible files, avoiding a no-op merge call.

resolvedPaths.Add(sourcePath)

Only add paths that actually exist on disk. Missing files are added to SkippedFiles and logged but do not abort the whole merge -- other files proceed.

7.4.4 Helper methods in FolderService

```

private static string ResolveSourcePath(string filePath, string appBaseDir) =>
    Path.IsPathRooted(filePath) ? filePath : Path.Combine(appBaseDir, filePath);

private void EnsureFolder(string path, FolderResult result)
{
    if (!Directory.Exists(path))
    {
        Directory.CreateDirectory(path);
        result.FoldersCreated++;
        _log.Info($"Created folder: {path}");
    }
}

```

```

}

private string BuildFullPath(string employeePath, string folderPath, FolderResult result)
{
    string[] segments = folderPath
        .Replace('/', Path.DirectorySeparatorChar)
        .Replace('\\', Path.DirectorySeparatorChar)
        .Split(Path.DirectorySeparatorChar, StringSplitOptions.RemoveEmptyEntries)
        .Select(FileNamingHelper.SanitizeSegment)
        .ToArray();

    string current = employeePath;
    foreach (string segment in segments)
    {
        current = Path.Combine(current, segment);
        EnsureFolder(current, result);
    }
    return current;
}

```

Path.IsPathRooted(filePath) ? filePath : Path.Combine(appBaseDir, filePath)

If FilePath is absolute (starts with 'C:\' or '/') use it as-is. Otherwise treat it as relative to the executable's folder. This allows the dummy data to work whether the app is run from VS, CLI, or as a published exe.

.Replace('/', Path.DirectorySeparatorChar).Replace("\", Path.DirectorySeparatorChar)

Normalise both forward and backward slashes to the OS separator. Path.DirectorySeparatorChar is '\' on Windows, '/' on Linux/Mac.

.Split(Path.DirectorySeparatorChar, StringSplitOptions.RemoveEmptyEntries)

Split on the separator. RemoveEmptyEntries prevents empty strings if the path starts or ends with a slash.

.Select(FileNamingHelper.SanitizeSegment)

Method group syntax -- equivalent to '.Select(s => FileNamingHelper.SanitizeSegment(s))'. Sanitises each path segment independently so only the individual segment's invalid chars are replaced, not the whole path.

string current = employeePath; foreach (segment) { current = Combine(...); EnsureFolder...
}

Incrementally build the path segment by segment, creating each directory as we go. This handles arbitrarily deep paths including 3-level paths like '9. Performance Appraisal/9.2 Current Year/9.2.1 Mid-Year'.

7.5 PdfMergeService.cs

PdfMergeService uses the PdfSharp library to combine multiple PDFs and images into a single output PDF. PdfSharp is a well-known .NET open-source library that manipulates PDF files without needing Adobe Acrobat.

```
private static readonly HashSet<string> EligibleExtensions =
    new(StringComparer.OrdinalIgnoreCase)
    {
        ".pdf", ".jpg", ".jpeg", ".jif", ".png",
        ".bmp", ".tiff", ".tif", ".webp"
    };

public PdfMergeService(LogHelper log) => _log = log;

public static bool IsEligible(string filePath) =>
    EligibleExtensions.Contains(Path.GetExtension(filePath));

public void Merge(IEnumerable<string> sourcePaths, string outputPath)
{
    using var output = new PdfDocument();
    foreach (string sourcePath in sourcePaths)
    {
        string ext = Path.GetExtension(sourcePath);
        try {
            if (ext.Equals(".pdf", StringComparison.OrdinalIgnoreCase))
                ImportPdfPages(sourcePath, output);
            else
                AppendImagePage(sourcePath, output);
        }
        catch (Exception ex) {
            _log.Warn($"Skipping '{Path.GetFileName(sourcePath)}': {ex.Message}");
        }
    }
    if (output.PageCount == 0)
        throw new InvalidOperationException(...);
    output.Save(outputPath);
}
```

HashSet<string> EligibleExtensions = new(StringComparer.OrdinalIgnoreCase)

A HashSet for O(1) lookups (much faster than a List for Contains). StringComparer.OrdinalIgnoreCase makes '.PDF' match '.pdf'. The set is declared with a collection initialiser -- the values in braces are added at construction.

public static bool IsEligible(string filePath)

'static' means you can call it without creating an instance: 'PdfMergeService.IsEligible(path)'. It simply checks if the file's extension is in the set.

using var output = new PdfDocument();

'using' ensures Dispose() is called automatically when the block exits -- even if an exception occurs. PdfDocument implements IDisposable. This prevents file handle leaks. Think of it like Python's 'with open(...)'.

try { ... } catch (Exception ex) { _log.Warn(...); }

Per-file error handling. If one file fails to merge (corrupt, locked, etc.), log a warning and continue. The loop does NOT stop -- remaining files are still processed. Only after the loop do we check if 0 pages were produced.

if (output.PageCount == 0) throw new InvalidOperationException(...)

Guard: if EVERY file failed, the merged PDF would have zero pages which is invalid. Throw so FolderService's outer try-catch can log the failure and add it to SkippedFiles rather than saving an empty PDF.

output.Save(outputPath)

Write the complete in-memory PdfDocument to disk as a real PDF file.

7.5.1 ImportPdfPages

```
private static void ImportPdfPages(string sourcePath, PdfDocument output)
{
    using PdfDocument input = PdfReader.Open(sourcePath,
                                              PdfDocumentOpenMode.Import);

    foreach (PdfPage page in input.Pages)
        output.AddPage(page);
}
```

Opens the source PDF in Import mode (read-only, optimised for page extraction). Iterates every page and adds a copy to the output document. The 'using' declaration (C# 8+) keeps the scope until end of the enclosing block.

7.5.2 AppendImagePage

```
private static void AppendImagePage(string sourcePath, PdfDocument output)
{
    using XImage image = XImage.FromFile(sourcePath);

    bool landscape = image.PixelWidth > image.PixelHeight;
    PdfPage page = output.AddPage();
    page.Width = XUnit.FromMillimeter(landscape ? 297 : 210);
    page.Height = XUnit.FromMillimeter(landscape ? 210 : 297);

    using XGraphics gfx = XGraphics.FromPdfPage(page);

    double scaleX = page.Width.Point / image.PixelWidth;
    double scaleY = page.Height.Point / image.PixelHeight;
    double scale = Math.Min(scaleX, scaleY);
    double w = image.PixelWidth * scale;
    double h = image.PixelHeight * scale;
    double x = (page.Width.Point - w) / 2;
    double y = (page.Height.Point - h) / 2;

    gfx.DrawImage(image, x, y, w, h);
}
```

bool landscape = image.PixelWidth > image.PixelHeight;

If the image is wider than it is tall, it's landscape orientation. We then create a page that is 297mm wide and 210mm tall (A4 landscape). Otherwise standard A4 portrait: 210mm x 297mm.

page.Width = XUnit.FromMillimeter(landscape ? 297 : 210);

XUnit is PdfSharp's unit-aware wrapper. FromMillimeter converts to Points internally (1 inch = 72 points; 1 mm ≈ 2.835 points). The ternary selects the right dimension.

double scaleX = page.Width.Point / image.PixelWidth;

Calculate the scale factor to fit the image width to the page width in Points. Similarly for height.

double scale = Math.Min(scaleX, scaleY);

Use the smaller scale factor so the image fits within both dimensions while maintaining aspect ratio. This is 'fit-inside' scaling.

double x = (page.Width.Point - w) / 2;

Centre the image horizontally. Total page width minus image width divided by 2 gives the left margin needed for centring.

gfx.DrawImage(image, x, y, w, h);

Render the image onto the PDF page at position (x,y) with dimensions (w,h). This is a vector-PDF draw call, not a bitmap paste -- the image is embedded.

7.6 ReportService.cs

```
public class ReportService
{
    public void WriteSummary(string summaryFilePath, DateTime runTime,
                            FolderResult result)
    {
        var sb = new StringBuilder();
        sb.AppendLine("DumpUtility Run Summary");
        sb.AppendLine("=====");
        sb.AppendLine($"Run Timestamp    : {runTime:yyyy-MM-dd HH:mm:ss}");
        sb.AppendLine($"Employees      : {result.EmployeePaths.Count}");
        ...
        foreach (var (folder, path) in result.EmployeePaths)
            sb.AppendLine($" {folder} -> {path}");
        ...
        foreach (var (file, error) in result.SkippedFiles)
            sb.AppendLine($" {file} - {error}");

        File.WriteAllText(summaryFilePath, sb.ToString());
    }
}
```

var sb = new StringBuilder();

StringBuilder is a mutable string buffer -- efficient for building strings by appending many pieces. Using '+' concatenation in a loop would create a new string object every iteration, which is slow.

sb.AppendLine(...)

Adds text followed by the platform line ending (\r\n on Windows). Equivalent to sb.Append(text + Environment.NewLine).

foreach (var (folder, path) in result.EmployeePaths)

Iterating a Dictionary<string,string> with tuple destructuring. Each iteration gives a KeyValuePair; the '(folder, path)' deconstructs it into two named variables. Equivalent to 'foreach (var kvp in ...) { var folder = kvp.Key; ... }'.

foreach (var (file, error) in result.SkippedFiles)

Same pattern for the list of value tuples. Each element is a (string, string) tuple; the names file and error are the element labels declared in FolderResult.

File.WriteAllText(summaryFilePath, sb.ToString())

Creates (or overwrites) the file and writes the entire StringBuilder content as a single string. Simpler than using StreamWriter for small files.

8. Program.cs -- The Entry Point

Program.cs uses C# 9 'top-level statements' -- there is no explicit class or Main method. The compiler generates one automatically. Code runs sequentially from top to bottom. Think of it exactly like a Python script.

```
using DumpUtility.Helpers;
using DumpUtility.Services;

string appBaseDir = AppContext.BaseDirectory;
string configPath = Path.Combine(appBaseDir, "config.json");
DateTime runTime = DateTime.Now;
string timestamp = runTime.ToString("ddMMyyyy_HHmm");
```

using DumpUtility.Helpers; using DumpUtility.Services;

Import the namespaces so we can write 'LogHelper' instead of 'DumpUtility.Helpers.LogHelper' everywhere.

string appBaseDir = AppContext.BaseDirectory;

The folder where the compiled exe lives. When you run 'dotnet run', this is the bin/Debug/net8.0/ folder. When you run a published exe, it's the publish/ folder. Using this ensures config.json and dummy_data/ are always found relative to the exe, regardless of where the terminal's current directory is.

DateTime runTime = DateTime.Now;

Capture the start time ONCE so all filenames in this run share the same timestamp. If we called DateTime.Now multiple times, names could differ by a second.

string timestamp = runTime.ToString("ddMMyyyy_HHmm");

Custom date format: day-month-year_hour-minute. E.g. 26052026_1430. Used in folder, log, and summary file names.

```
var configService = new ConfigService();
DumpUtility.Models.AppConfig config;

try
{
    config = configService.Load(configPath);
}
catch (Exception ex)
{
    Console.WriteLine($"[ERROR] Failed to load config.json: {ex.Message}");
    Environment.Exit(1);
    return;
}
```

DumpUtility.Models.AppConfig config;

Declare the variable WITHOUT initialising it. The try block sets it. C# requires all variables to be assigned before use -- the compiler verifies this.

catch (Exception ex)

'Exception' is the base type for all exceptions. Catching it here means any error from Load() is caught, including FileNotFoundException, InvalidOperationException, etc.

Environment.Exit(1);

Terminates the process immediately with exit code 1 (convention: non-zero = failure). Exit code 0 means success. CI/CD systems and shell scripts check this value.

return;

The compiler doesn't know that Exit(1) never returns. The 'return' is there to satisfy the compiler that 'config' is definitely assigned on all code paths after the try-catch. Without it, the 'config is null' warning appears further down.

```
Directory.CreateDirectory(config.BaseFolder);
string logFilePath = Path.Combine(config.BaseFolder, $"Log_{timestamp}.txt");
var log = new LogHelper(logFilePath);

log.Info("App started");
log.Info($"Config loaded. BaseFolder: {config.BaseFolder}. DB config present:
{!string.IsNullOrEmpty(config.Database?.Server)}");
```

Directory.CreateDirectory(config.BaseFolder)

Creates the BaseFolder (e.g. C:\DumpOutput) if it doesn't exist. CreateDirectory is idempotent -- if the folder already exists, it does nothing.

var log = new LogHelper(logFilePath)

Construct the logger. From this point on, log.Info/Warn/Error write to the log file.

config.Database?.Server

Null-conditional operator '?.'. If config.Database is null, the whole expression returns null instead of throwing a NullReferenceException. Safe navigation operator -- similar to optional chaining in JavaScript (obj?.prop).

```
string recordsPath = Path.Combine(appBaseDir, "dummy_data", "records2.json");
IDataService dataService = new DummyDataService(recordsPath);
var records = dataService.GetRecords().ToList();

string runFolderName = $"Dump_Utility_{timestamp}";
string runFolderPath = Path.Combine(config.BaseFolder, runFolderName);

var mergeService = new PdfMergeService(log);
var folderService = new FolderService(log, mergeService);
var result = folderService.Execute(runFolderPath, records, appBaseDir);

string summaryPath = Path.Combine(config.BaseFolder, $"Summary_{timestamp}.txt");
var reportService = new ReportService();
reportService.WriteSummary(summaryPath, runTime, result);
```

IDataService dataService = new DummyDataService(recordsPath);

Key line! The variable is typed as IDataService (the interface), not DummyDataService (the concrete class). This is the abstraction point: to switch to real database data, change ONLY this one line.

dataService.GetRecords().ToList()

.ToList() materialises the lazy IEnumerable into a List<DocumentRecord>. This forces all records to be loaded from disk NOW, before we start creating folders. It also allows .Count to work without re-reading the file.

var mergeService = new PdfMergeService(log);

PdfMergeService needs the logger to report skipped corrupt files. We create it here and pass it into FolderService rather than FolderService creating it internally -- this is manual dependency injection.

var result = folderService.Execute(runFolderPath, records, appBaseDir);

This single call does all the heavy lifting: creates folders, copies files, merges PDFs. It blocks until complete and returns a FolderResult summary.

```
var duration = DateTime.Now - runTime;
log.Info($"App      finished.      Duration:      {duration:hh\\:mm\\:ss}.      Files      written:
{result.FilesWritten}...");

Console.WriteLine("Complete.");
Console.WriteLine($"  Employees  : {result.EmployeePaths.Count}");
Console.WriteLine($"  Folders   : {result.FoldersCreated}");
Console.WriteLine($"  Files     : {result.FilesWritten}");
Console.WriteLine($"  Merged    : {result.MergedGroups} group(s)...");
Console.WriteLine($"  Skipped   : {result.SkippedFiles.Count}");
```

'DateTime.Now - runTime' subtracts two DateTime values, producing a TimeSpan. The format ':hh\\:mm\\:ss' formats it as HH:MM:SS. The double-backslash is needed because the format string is inside an interpolated string -- one backslash escapes the colon for the TimeSpan formatter.

9. Test Cases -- PdfMergeServiceTests.cs

Tests are in the 'DumpUtility.Tests' project which uses the xUnit testing framework -- the most popular .NET test framework. Run them with:

```
dotnet test
```

The test class implements IDisposable -- this means it has a Dispose() method that xUnit calls after each test to clean up temporary files.

9.1 Test Fixture Setup (Constructor + Dispose)

```
public PdfMergeServiceTests()
{
    _tmpDir = Path.Combine(Path.GetTempPath(), $"PdfMergeTests_{Guid.NewGuid():N}");
    Directory.CreateDirectory(_tmpDir);
    _logPath = Path.Combine(_tmpDir, "test.log");
    _svc = new PdfMergeService(new LogHelper(_logPath));
}

public void Dispose() => Directory.Delete(_tmpDir, recursive: true);
```

Each test gets its own fresh temporary directory (e.g. 'C:\Temp\PdfMergeTests_abc123...'). Guid.NewGuid() generates a globally unique ID ensuring no two test runs collide. Dispose() deletes the entire temp folder after each test. This makes tests fully isolated -- they cannot affect each other or leave garbage on disk.

9.2 Inline BMP Data (MinimalBmpBytes, LandscapeBmpBytes)

```
private static readonly byte[] MinimalBmpBytes = {
    0x42, 0x4D, ... (58 bytes total)
};
```

These are hardcoded binary BMP files in byte array form. '0x42 0x4D' is the BMP magic number ('BM' in ASCII). Using raw bytes avoids having to ship test image files with the repo -- the test is entirely self-contained. MinimalBmpBytes is a 1x1 white pixel. LandscapeBmpBytes is a 4x1 pixel image (wider than tall) to test landscape page orientation.

9.3 Fixture Helper Methods

MakePdf(pageCount, name?) -- Creates a real PdfSharp PDF with N blank pages in the temp dir. Used as valid input.

MakeBmp(name?) -- Writes MinimalBmpBytes to a .bmp file in the temp dir.

MakeLandscapeBmp(name?) -- Writes LandscapeBmpBytes -- a wider-than-tall image.

MakeCorrupt(ext, name?) -- Writes the text 'this is not a valid file' with the given extension. PdfSharp will fail to parse it.

PageCount(pdfPath) -- Opens a PDF and returns its page count. Used to ASSERT on merge results.

9.4 Test: IsEligible_CorrectForAllExtensions

```
[Theory]
[InlineData(".pdf", true)]
[InlineData(".docx", false)]
...
```

```
public void IsEligible_CorrectForAllExtensions(string ext, bool expected)
{
    Assert.Equal(expected, PdfMergeService.IsEligible($"file{ext}"));
}
```

[Theory] + [InlineData] is xUnit's parameterised test. One test method runs MULTIPLE times, once per [InlineData] row, with different arguments. This is equivalent to writing 12 separate test methods. Assert.Equal(expected, actual) fails the test if the values differ, printing both values in the failure message.

9.5 Test: IsEligible_CaseInsensitive

```
[Theory]
[InlineData(".PDF")]
[InlineData(".JPG")]
[InlineData(".BMP")]
public void IsEligible_CaseInsensitive(string ext)
{
    Assert.True(PdfMergeService.IsEligible($"file{ext}"));
}
```

Verifies that the HashSet uses StringComparer.OrdinalIgnoreCase correctly. Assert.True is the simplest assertion -- it just checks the value is true.

9.6 Test: SinglePdf_ProducesExactPageCount

```
[Fact]
public void SinglePdf_ProducesExactPageCount()
{
    string src    = MakePdf(1);
    string output = Path.Combine(_tmpDir, "out.pdf");

    _svc.Merge([src], output);

    Assert.Equal(1, PageCount(output));
}
```

[Fact] marks a simple non-parameterised test. ARRANGE: create a 1-page PDF. ACT: merge it (a list of one item). ASSERT: output has exactly 1 page. This follows the AAA (Arrange-Act-Assert) pattern.

9.7 Test: MultiPdf_PageCountsSum

```
[Fact]
public void MultiPdf_PageCountsSum()
{
    string src1  = MakePdf(2);
    string src2  = MakePdf(3);
    string output = Path.Combine(_tmpDir, "out.pdf");

    _svc.Merge([src1, src2], output);

    Assert.Equal(5, PageCount(output));
}
```

Merging a 2-page and a 3-page PDF should produce a 5-page PDF. Validates that ALL pages from ALL input files are included in the merge output.

9.8 Test: SingleImage_ProducesOnePage

Merging one BMP image should produce a 1-page PDF (one page per image is the rule). This tests the AppendImagePage code path.

9.9 Test: PdfAndImage_ProducesSummedPages

Merging a 2-page PDF and 1 image should produce 3 pages total. Tests the mixed content path -- both ImportPdfPages and AppendImagePage are exercised.

9.10 Test: CorruptFile_IsSkippedMergeSucceeds

```
[Fact]
public void CorruptFile_IsSkippedMergeSucceeds()
{
    string valid    = MakePdf(1);
    string corrupt = MakeCorrupt(".pdf");
    string output   = Path.Combine(_tmpDir, "out.pdf");

    _svc.Merge([valid, corrupt], output);

    Assert.Equal(1, PageCount(output));
    Assert.Contains("WARN", File.ReadAllText(_logPath));
}
```

Validates the fault-tolerance behaviour: when a corrupt file is included, the valid files should still produce output. The corrupt file should be SKIPPED with a WARN log entry (not ERROR -- it's expected behaviour). Assert.Contains checks that the log file contains the string 'WARN'.

9.11 Test: AllFilesFail_Throws

```
[Fact]
public void AllFilesFail_Throws()
{
    string c1 = MakeCorrupt(".pdf", "c1.pdf");
    string c2 = MakeCorrupt(".pdf", "c2.pdf");

    Assert.Throws<InvalidOperationException>(() => _svc.Merge([c1, c2], output));
}
```

Assert.Throws<T>(() => ...) asserts that the lambda throws the specified exception type. The '() => ...' is a C# lambda (anonymous function). When ALL files fail, zero pages are produced and Merge() MUST throw -- saving an empty PDF would be a silent failure.

9.12 Test: LandscapeImage_ProducesPageWiderThanTall

```
[Fact]
public void LandscapeImage_ProducesPageWiderThanTall()
{
    string src    = MakeLandscapeBmp();
    string output = Path.Combine(_tmpDir, "out.pdf");

    _svc.Merge([src], output);

    using var doc = PdfReader.Open(output, PdfDocumentOpenMode.Import);
}
```

```
PdfPage page = doc.Pages[0];  
Assert.True(page.Width.Point > page.Height.Point);  
}
```

Opens the output PDF and reads the first page's dimensions. Asserts that Width > Height, confirming the landscape detection and page orientation logic in `AppendImagePage` works correctly.

10. Database View Integration (Future Requirement)

The manager will provide a SQL Server VIEW that returns the same four columns as records.json. This section explains exactly what you need to change and add.

10.1 What a SQL VIEW is

A VIEW is a saved SQL query stored in the database. Querying a VIEW looks exactly like querying a table: 'SELECT * FROM vw_HRDocuments'. The DBA creates the view; you just read from it. The data comes back as rows with columns, identical in structure to the JSON:

```
-- Example of what the VIEW might look like (DBA writes this, not you):
CREATE VIEW vw_HRDocuments AS
SELECT
    e.EmployeeId,
    e.EmployeeName,
    d.FolderPath,
    d.FilePath
FROM Employees e
JOIN Documents d ON e.EmployeeId = d.EmployeeId
WHERE d.IsActive = 1;
```

10.2 Step 1 -- Add NuGet package

You need Microsoft.Data.SqlClient to talk to SQL Server from .NET:

```
# Run from the DumpUtility/ directory:
dotnet add package Microsoft.Data.SqlClient
```

10.3 Step 2 -- Update config.json

```
{
  "BaseFolder": "C:\\\\DumpOutput",
  "Database": {
    "Server": "SQLSERVER01",
    "DatabaseName": "HRDB",
    "Username": "hrapp",
    "Password": "yourpassword",
    "Port": 1433
  }
}
```

The DatabaseConfig class already has all these fields. The ConfigService already reads them. Nothing to change in those classes.

10.4 Step 3 -- Write SqlDataService.cs

Create a new file: DumpUtility/Services/SqlDataService.cs

```
using System.Data;
using Microsoft.Data.SqlClient;
using DumpUtility.Models;

namespace DumpUtility.Services;
```



```

public class SqlDataService : IDataService
{
    private readonly DatabaseConfig _db;

    public SqlDataService(DatabaseConfig db) => _db = db;

    public IEnumerable<DocumentRecord> GetRecords()
    {
        string connStr = $"Server={_db.Server},{_db.Port};" +
            $"Database={_db.DatabaseName};" +
            $"User Id={_db.Username};Password={_db.Password};" +
            "TrustServerCertificate=True;";

        var records = new List<DocumentRecord>();

        using var conn = new SqlConnection(connStr);
        conn.Open();

        using var cmd = new SqlCommand(
            "SELECT EmployeeId, EmployeeName, FolderPath, FilePath" +
            " FROM vw_HRDocuments", conn);

        using SqlDataReader reader = cmd.ExecuteReader();
        while (reader.Read())
        {
            records.Add(new DocumentRecord
            {
                EmployeeId = reader.GetInt32("EmployeeId"),
                EmployeeName = reader.GetString("EmployeeName"),
                FolderPath = reader.GetString("FolderPath"),
                FilePath = reader.GetString("FilePath"),
            });
        }

        return records;
    }
}

```

public class SqlDataService : IDataService

Implements the SAME interface as DummyDataService. The rest of the codebase never knows the difference.

string connStr = \$"Server=..."

ADO.NET connection string format for SQL Server. Each part is a key=value pair separated by semicolons. TrustServerCertificate=True is needed if the server uses a self-signed certificate (common in corporate environments).

using var conn = new SqlConnection(connStr); conn.Open();

SqlConnection represents a database connection. Open() establishes the TCP connection. The 'using' ensures Close() is called automatically when done.

using var cmd = new SqlCommand("SELECT ... FROM vw_HRDocuments", conn);

SqlCommand holds the SQL query to execute. Replace 'vw_HRDocuments' with the actual view name when the DBA provides it.

```
using SqlDataReader reader = cmd.ExecuteReader();
```

ExecuteReader sends the query and returns a forward-only cursor. Each call to reader.Read() advances to the next row, returning false when done.

```
reader.GetInt32("EmployeeId"), reader.GetString("EmployeeName")
```

Type-safe column readers. GetInt32 reads an integer column, GetString reads a text column. The column name must match what the VIEW returns exactly.

10.5 Step 4 -- One line change in Program.cs

Find this line:

```
IDataService dataService = new DummyDataService(recordsPath);
```

Replace with:

```
IDataService dataService = new SqlDataService(config.Database);
```

That is the **ONLY** change needed in Program.cs. Because the variable is typed as IDataService, FolderService receives the same interface and never needs to change. This is the benefit of designing with interfaces from the start.

10.6 File paths from the database

The current dummy data uses relative paths like 'dummy_data/pdf/rishikesh.pdf'. The real database VIEW will likely return ABSOLUTE paths like '\\FILESERVER\HR\123\cv.pdf' or 'C:\HR\files\cv.pdf'. FolderService.ResolveSourcePath already handles this:

```
private static string ResolveSourcePath(string filePath, string appBaseDir) =>  
    Path.IsPathRooted(filePath) ? filePath : Path.Combine(appBaseDir, filePath);
```

If FilePath is absolute (starts with a drive letter or UNC path), it is used as-is. No code change needed for absolute paths from the database.

10.7 Error handling additions

The SqlDataService should also handle connection failures gracefully. The outer try-catch in Program.cs will catch any SqlException thrown by SqlDataService and log it as a fatal error -- so basic coverage exists already. For production you might add retry logic using Polly (a popular .NET resilience library).

Summary of changes for database integration

1. dotnet add package Microsoft.Data.SqlClient
 2. Fill in Database section in config.json with real server details
 3. Create DumpUtility/Services/SqlDataService.cs (code above)
 4. Change ONE line in Program.cs (new DummyDataService -> new SqlDataService)
 5. Replace 'vw_HRDocuments' with the actual view name from the DBA
- No other files need to change.

11. Glossary of C# / .NET Terms

.NET	Microsoft's free, open-source, cross-platform runtime and framework. Version 8 is the current LTS.
C#	The primary language for .NET. Statically typed, object-oriented, modern syntax.
NuGet	The .NET package manager. Like npm for Node or pip for Python.
PdfSharp	A NuGet package for creating and manipulating PDF files in .NET.
xUnit	A popular .NET unit testing framework. Tests are classes with [Fact] or [Theory] methods.
namespace	A named scope for organising classes. Declared at the top of a file.
class	A blueprint for objects. Contains properties and methods.
interface	A contract specifying method signatures that implementing classes must provide.
static	Belongs to the class itself, not to any instance. Called via ClassName.Method().
readonly	A field or property that can only be assigned once (in the constructor or declaration).
using	1) Imports a namespace. 2) Ensures Dispose() is called (resource management).
var	Type inference -- compiler figures out the type from the right-hand side.
string	Immutable sequence of Unicode characters. Use double quotes.
int	32-bit signed integer. Range: -2,147,483,648 to 2,147,483,647.
bool	Boolean: true or false.
List<T>	Generic list (dynamic array). T is the element type.
Dictionary<K, V>	Key-value map. O(1) lookup by key.
HashSet<T>	Set of unique values. O(1) Contains check.
IEnumerable<T>	The base interface for all sequences. Supports foreach.
LINQ	Language Integrated Query. Adds SQL-like operators (Where, GroupBy, OrderBy, Select, etc.) to any IEnumerable.
lambda	Anonymous function. Syntax: (params) => expression. E.g. r => r.EmployeeId.
?, ?.	? = nullable type annotation. ?. = null-conditional operator (safe navigation).
??	Null-coalescing operator. 'a ?? b' returns a if not null, otherwise b.
!	Null-forgiving operator. Tells the compiler 'I know this is not null.'
throw	Raises an exception, stopping execution and propagating up the call stack.
try/catch	try runs code; catch handles exceptions if they occur.
StringBuilder	Mutable string buffer. Efficient for appending many strings.
Path.Combine	Joins path segments using the OS directory separator. Safer than string concatenation.
File.Exists	Returns true if a file exists at the given path.
Directory.CreateDirectory	Creates a directory and all parent directories. Idempotent.
File.Copy(src, dst, overwrite)	Copies a file. overwrite=true replaces the destination if it exists.
File.AppendAllText	Opens file, appends text, closes file -- all in one call.
File.WriteAllText	Creates or overwrites a file with the given text content.
DateTime.Now	Current local date and time.
TimeSpan	A duration. Produced by subtracting two DateTimes.

Environment.Exit(n)	Terminates the process with exit code n. 0 = success, non-zero = error.
AppContext.BaseDirectory	The folder where the compiled application binary lives.
Guid.NewGuid()	Generates a globally unique identifier -- a 128-bit random value.
Expression body =>	Shorthand for a method or property with a single expression body.
Collection expression []	C# 12 syntax for creating empty or populated collections: [] or [a, b, c].
Value tuple (A, B)	A lightweight struct holding two or more values. No class needed.
Deconstruction	var (a, b) = tuple; -- unpacks a tuple or pair into named variables.
Target-typed new()	new() without repeating the type when the compiler can infer it from context.
RegexOptions.Compiled	Compiles the regex to IL/machine code for faster repeated use.

DumpUtility Codebase Explanation

AutoFileSys | Branch: rishikesh | Generated 2026-05-26